```
  const aiResponseStream = await getChainStream(input);
  let textContent = '';
  for await (const item of aiResponseStream) {
    textContent += item;
    stream.update(<ChatBubble role="assistant"
text={textContent} className={`mr-auto border-none`} />);
  }
  stream.done();
  aiState.done({
    ...aiState.get(),
    messages: [
      ...aiState.get().messages,
      {
        id: generateId(),
        role: 'assistant',
        content: textContent,
      },
    ],
  });

  return {
    id: generateId(),
    display: stream.value,
    role: 'assistant',
  };
}
```

Appends and displays chunks as they arrive

Saves the final response to conversation history

This setup chains multiple steps like data fetching, prompting, LLM processing, and parsing into one pipeline. By streaming, results flow to the UI piece by piece instead of waiting for completion, keeping the app responsive. The modular design also makes it easy to extend without rewriting the flow. This approach improves both flexibility and user experience.

Before we move on, there are a few caveats you need to be aware of when working with the two streams:

- *Closing the streams*—Close the streams after you finish working with them. In the provided code, this is done using `stream.done()` and `aiState.done()`.
- *Accumulating response*—Each item received from `aiResponseStream` represents a chunk of new data. To maintain a complete response, you must keep track of the current message using an accumulator like `textContent`.
- *Order of the chain and input variables*—Pay attention to the order of the chain and the input variables in the functions. In the provided example, the `city` variable in `chain.stream({ city })` is passed to `fetchWeatherData`, and the resulting object is then passed to the prompt template, which expects the rest of the template arguments. If an argument is missing, the LLM will likely complain.

6.2 Preparing and storing documents for retrieval using LangChain

With a basic integration of LangChain and the Vercel AI SDK in place, our focus now shifts to how we can prepare and store document embeddings for information